Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

A Shiny Deepa **1BM25EC400**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

Design and Verification of Combinational and Sequential Circuits using Verilog HDL

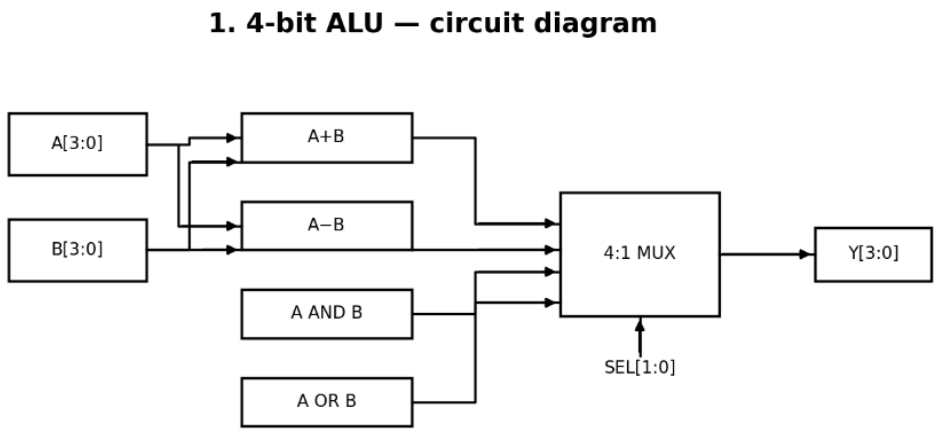
Introduction to EDA Playground

EDA Playground is a free, web-based platform used to write, simulate, and verify hardware description language (HDL) programs such as Verilog and SystemVerilog. It allows users to design digital circuits, write testbenches, execute simulations, and observe output waveforms without installing simulation software on their computers. It supports various simulators and provides a waveform viewer to analyze the behavior of digital circuits over time.

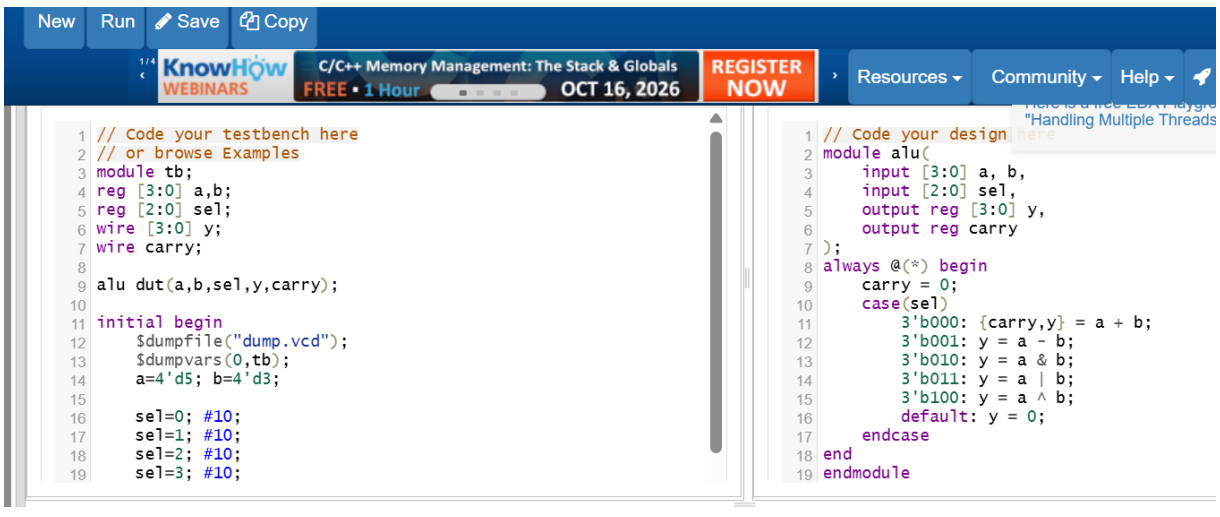
1.Arithmetic Logic Unit

An ALU is a combinational circuit that performs arithmetic and logical operations on binary inputs. The select lines determine which operation, such as addition, subtraction, AND, OR or XOR, is performed.

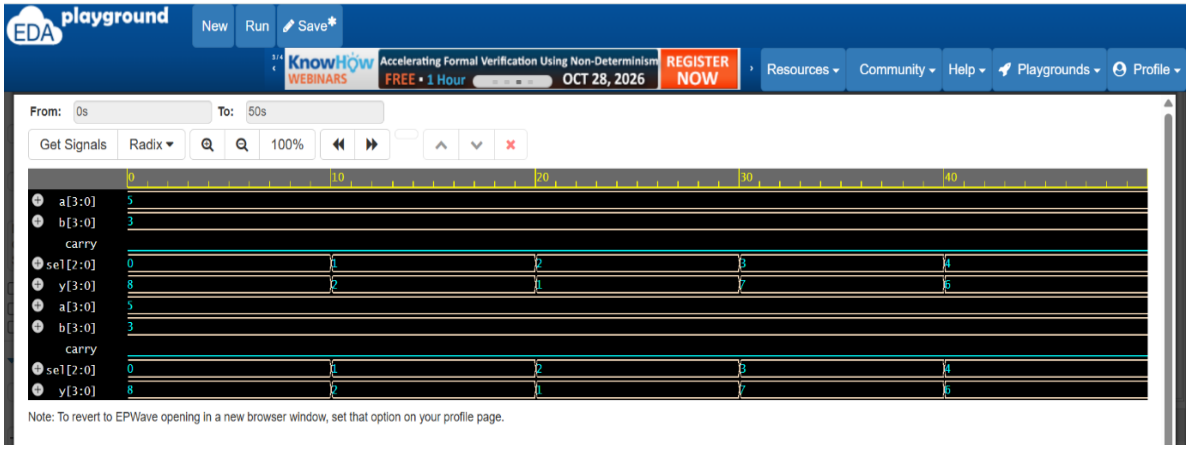
Circuit diagram



Code



Output Waveform

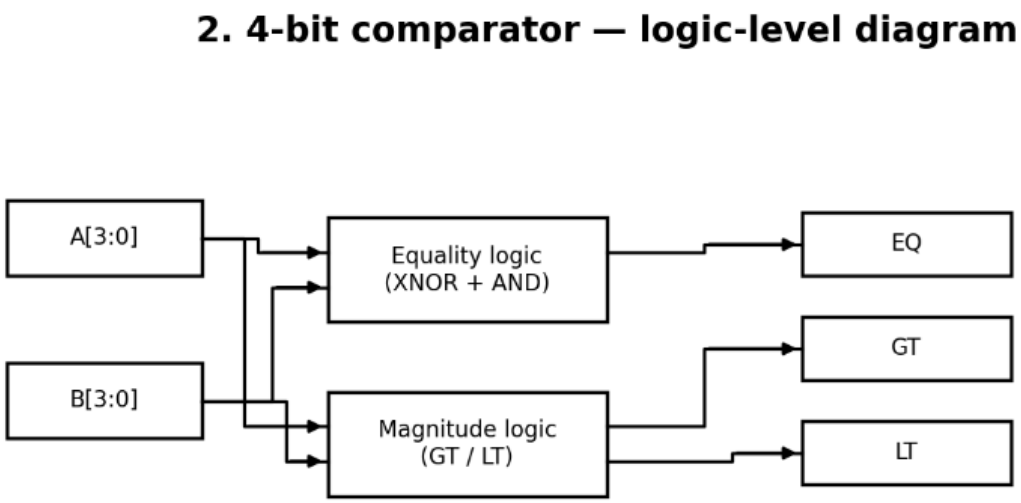


The waveform shows the input operands A and B along with the select lines that determine the operation. When the select lines change, the ALU performs the corresponding arithmetic or logical operation. The output Y displays the calculated result, while the carry output indicates a carry generated during addition. The waveform verifies that the ALU performs the selected operations correctly.

2. Comparator

A comparator is a combinational circuit used to compare two binary numbers. It produces three outputs indicating whether the first number is greater than, equal to or less than the second number.

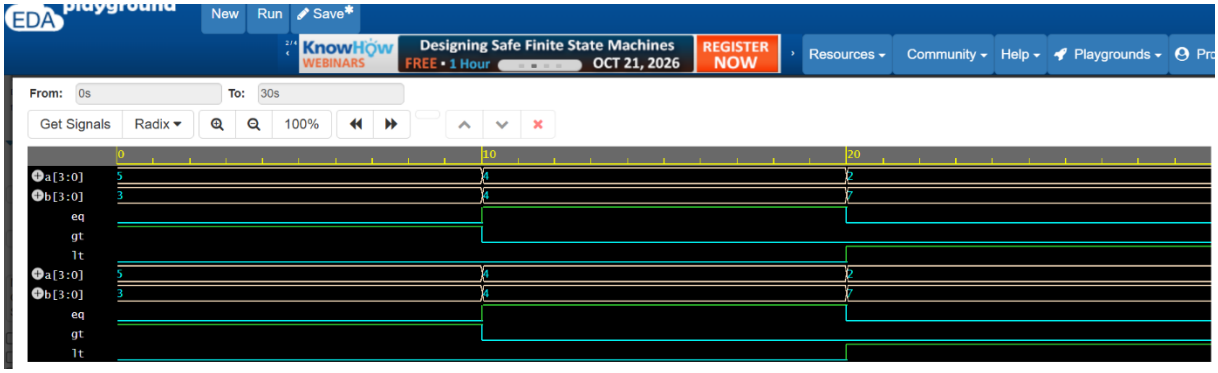
Logic Diagram



Code



Output waveform

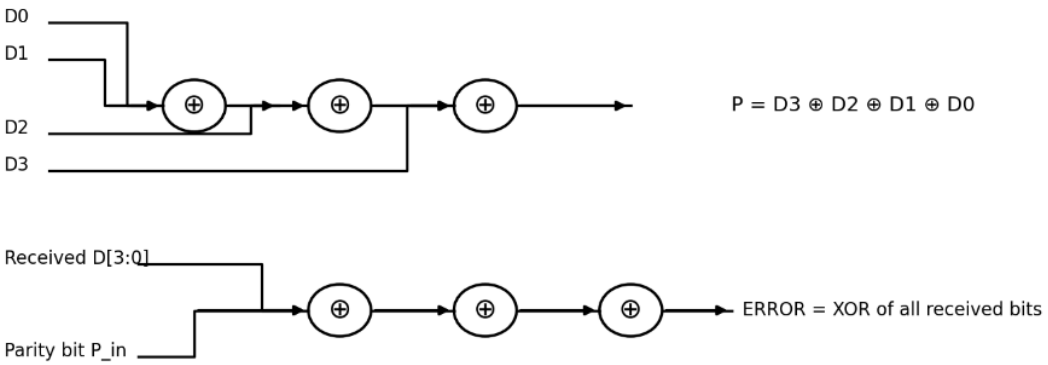


The waveform displays the two input numbers A and B and the corresponding GT, EQ and LT outputs. When A is greater than B, the GT output becomes HIGH. When both inputs are equal, EQ becomes HIGH, and when A is less than B, LT becomes HIGH. The waveform verifies that the comparator correctly identifies the relationship between the two binary numbers.

3. 4-Bit Parity Generator and Checker

A parity generator produces a parity bit using XOR operations to help detect errors in transmitted data. A parity checker verifies the received data and parity bit to identify parity errors.

3. 4-bit even parity generator and checker



Code



```
// Code your testbench here
// or browse Examples
module tb;
reg [3:0] data;
reg parity_in;
wire parity_out,error;

parity dut(data,parity_in,parity_out,error);

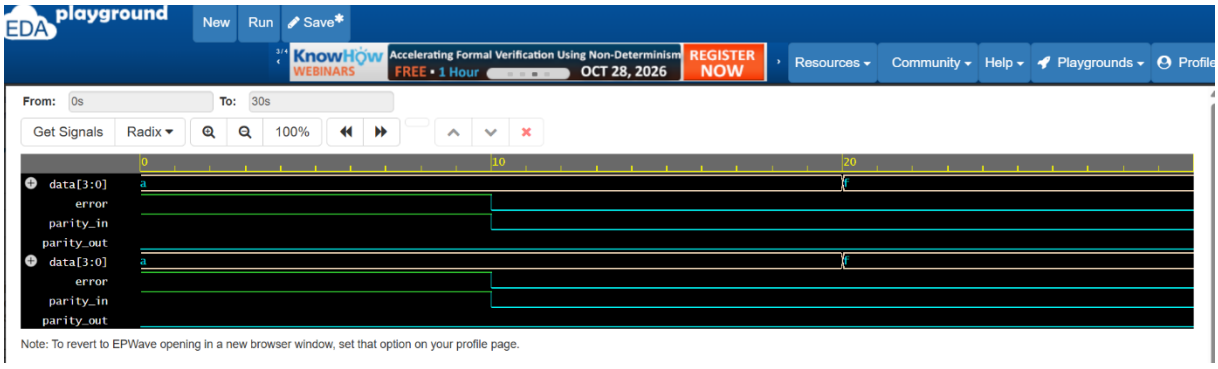
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);

    data=4'b1010; parity_in=1; #10;
    data=4'b1010; parity_in=0; #10;
    data=4'b1111; parity_in=0; #10;

    $finish;
end
```

```
// Code your design here
module parity(
    input [3:0] data,
    input parity_in,
    output parity_out,
    output error
);
assign parity_out = ^data;
assign error = ^{data,parity_in};
endmodule
```

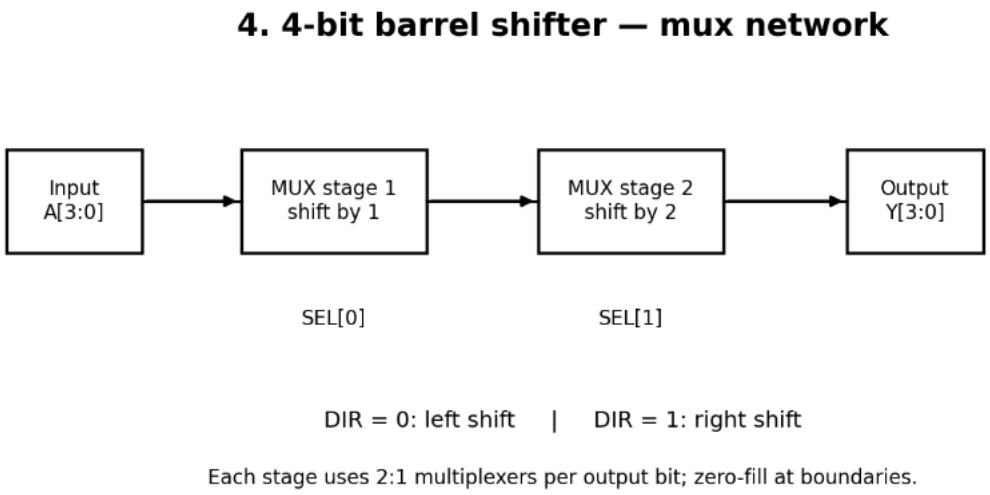
Output Waveform



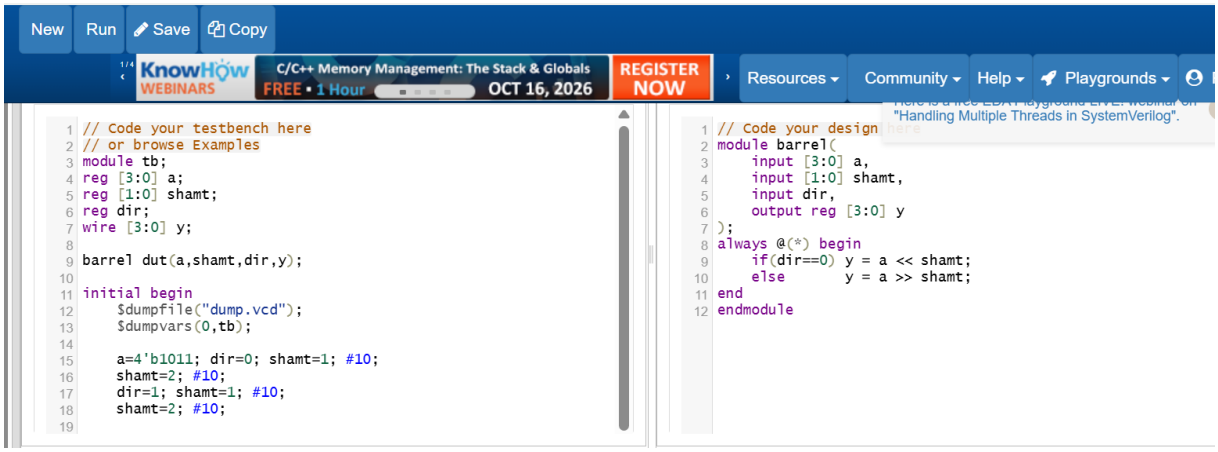
The waveform shows the four-bit input data, generated parity bit and error output. The parity generator calculates the parity bit using XOR operations on the input data. The error output becomes HIGH when the received data and parity bit produce an odd number of ones, indicating a parity error. The waveform demonstrates the generation of parity and detection of errors in the received data.

4. 4-Bit Barrel Shifter

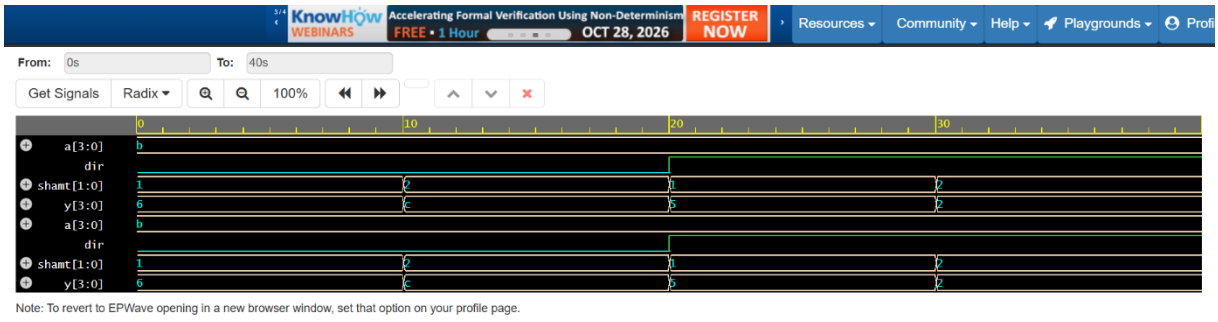
A barrel shifter is a combinational circuit that shifts binary data left or right by a specified number of positions. The direction and shift amount determine the resulting output.



Code



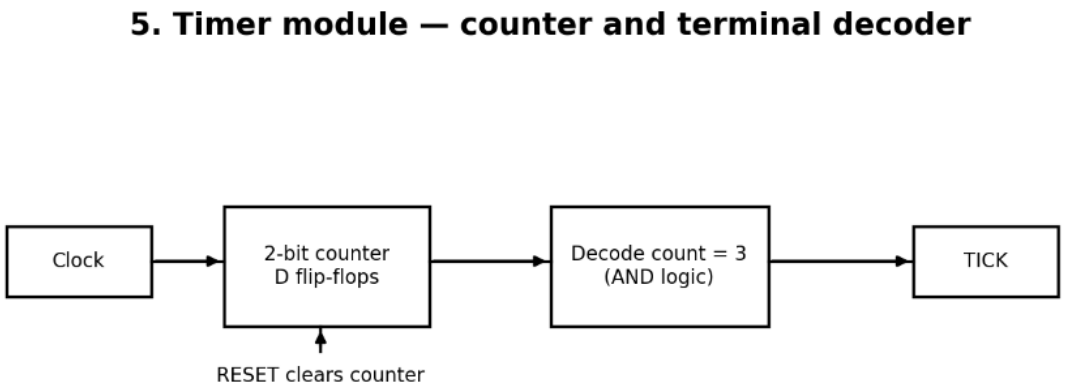
Output Waveforms



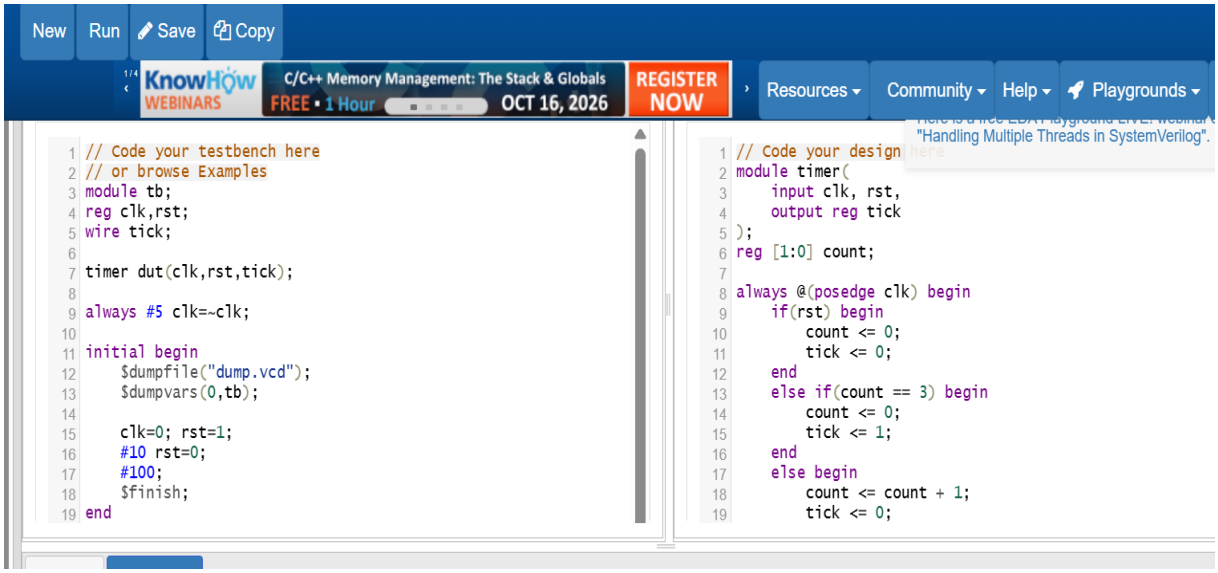
The waveform displays the input data, shift amount, direction control and output data. When the direction is set to left, the input bits shift towards the left by the specified number of positions. When the direction is set to right, the bits shift towards the right, with zeros filling the vacant positions. The waveform verifies that the barrel shifter performs the required shifting operation according to the control inputs.

5. Timer Module

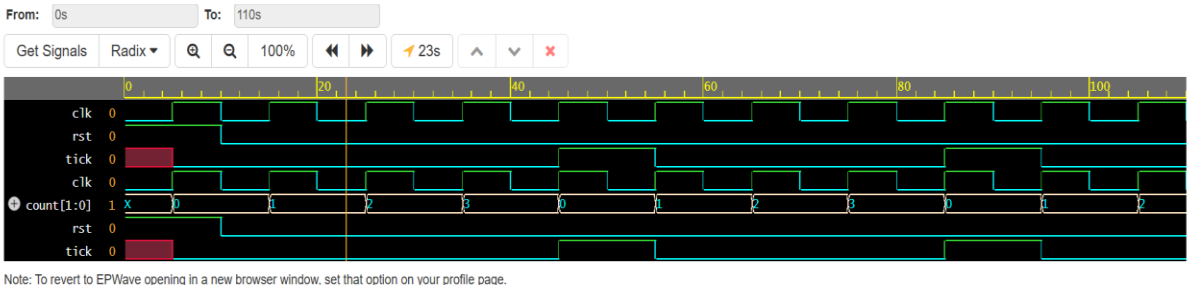
A timer module uses a clock-driven counter to measure a fixed number of clock cycles. It generates a tick pulse whenever the counter reaches its specified terminal count.



Code



Output Waveform

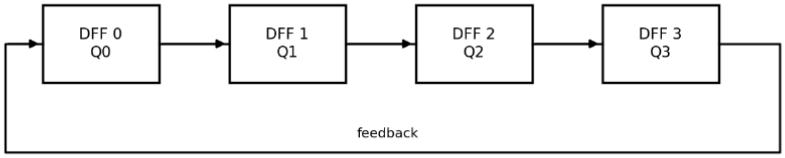


The waveform shows the clock, reset and tick output signals of the timer circuit. Initially, the reset signal clears the counter and keeps the tick output LOW. After reset is deactivated, the counter increments on every rising edge of the clock until it reaches the terminal count. The tick output becomes HIGH for one clock cycle every four clock cycles, demonstrating the correct operation of the timer.

6. Ring Counter

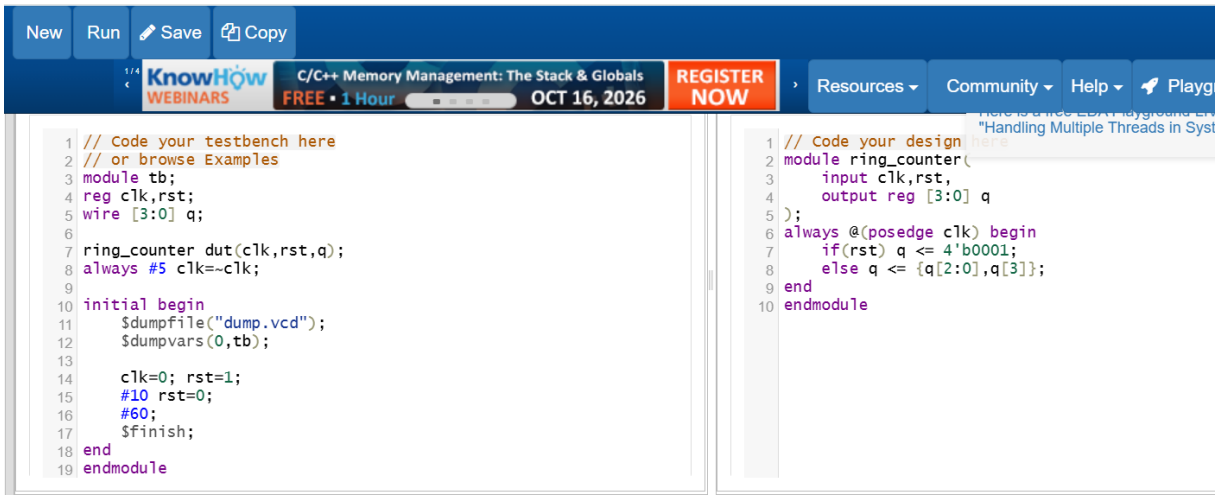
A ring counter consists of flip-flops connected in a circular shift arrangement. A single HIGH bit circulates through the flip-flops with every active clock edge.

6. 4-bit ring counter circuit

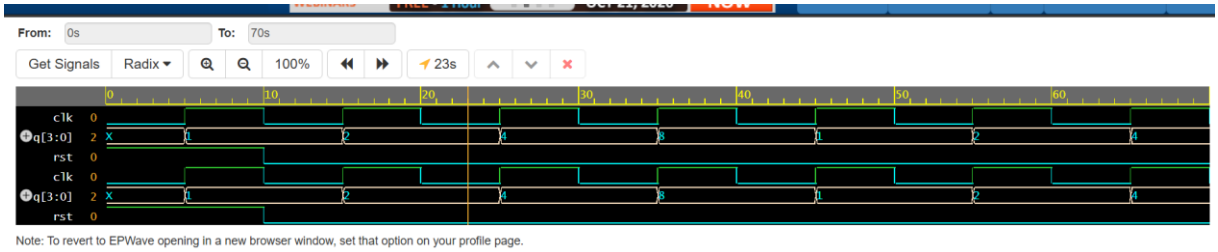


RESET initializes the state; outputs are Q[3:0].

Code



Output Waveform

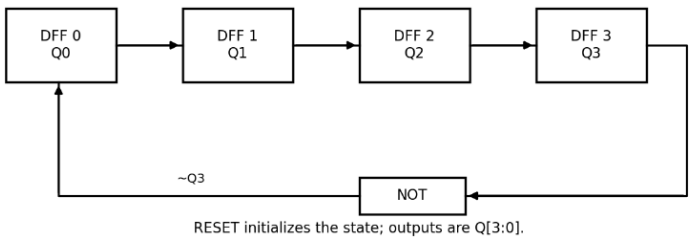


The waveform displays the clock, reset and four-bit counter output. Initially, reset loads the counter with the state 0001, in which only one bit is HIGH. On every rising clock edge, this HIGH bit shifts to the next position and circulates through the four flip-flops. The waveform shows the repeating sequence 0001, 0010, 0100 and 1000, verifying the operation of the ring counter.

7. Johnson Counter

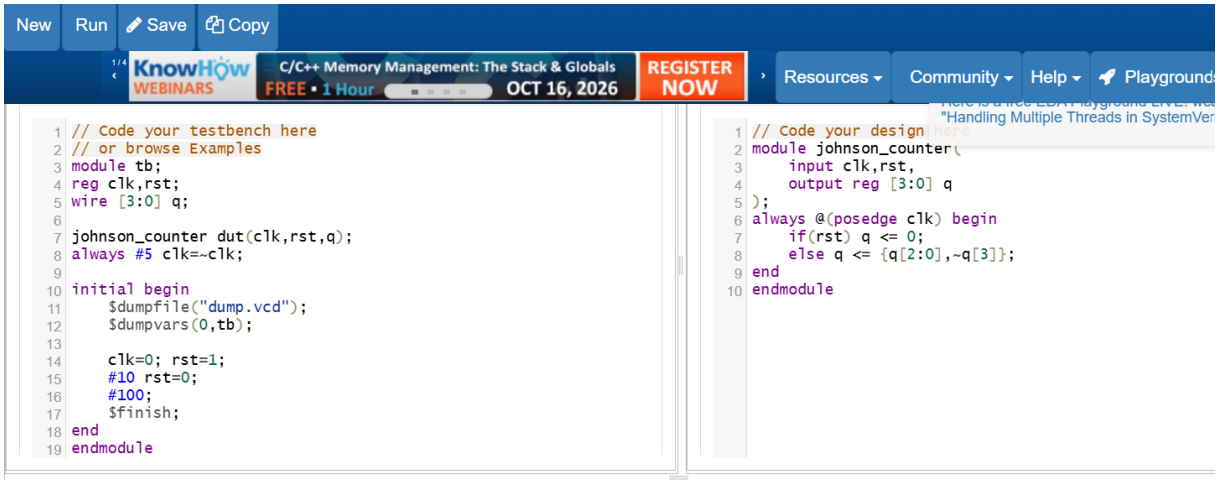
A Johnson counter is a shift-register counter in which the inverted output of the last flip-flop is fed back to the first flip-flop. A four-bit Johnson counter can generate up to eight distinct states.

7. 4-bit Johnson counter circuit

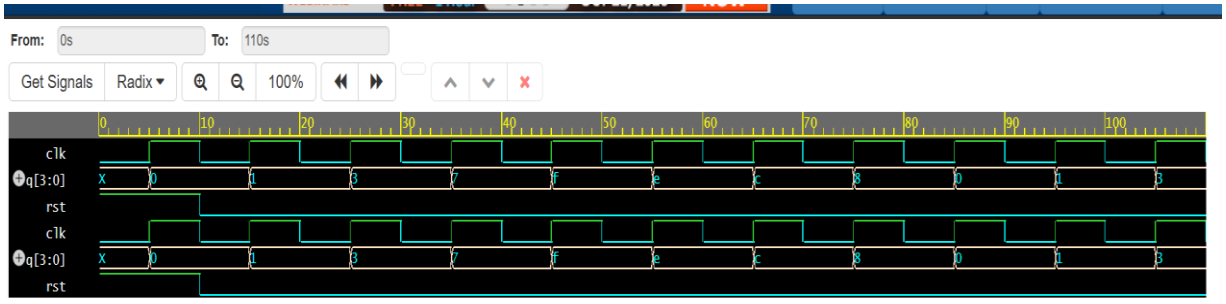


RESET initializes the state; outputs are Q[3:0].

Code



Output Waveforms



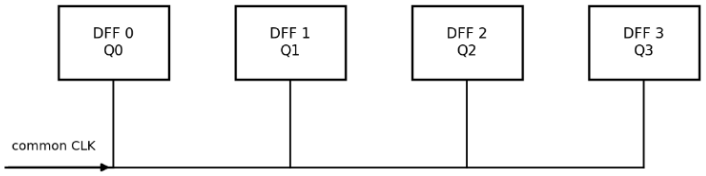
The waveform shows the clock, reset and four-bit output of the Johnson counter. Initially, reset clears all flip-flops, producing the state 0000. At every rising clock edge, the bits shift and the inverted output of the last flip-flop feeds the first stage. The waveform displays eight repeating states, verifying the correct operation of the four-bit Johnson counter.

8. Synchronous and Asynchronous Counters

Synchronous Counter

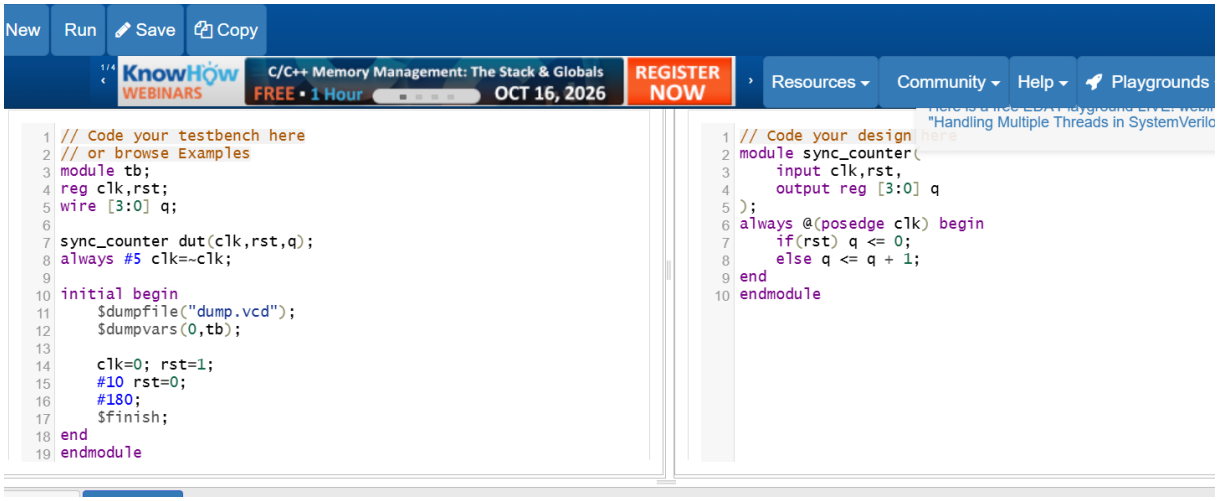
A synchronous counter consists of flip-flops driven by the same clock signal. A four-bit synchronous up counter increments its binary value on every rising clock edge.

8A. 4-bit synchronous counter circuit

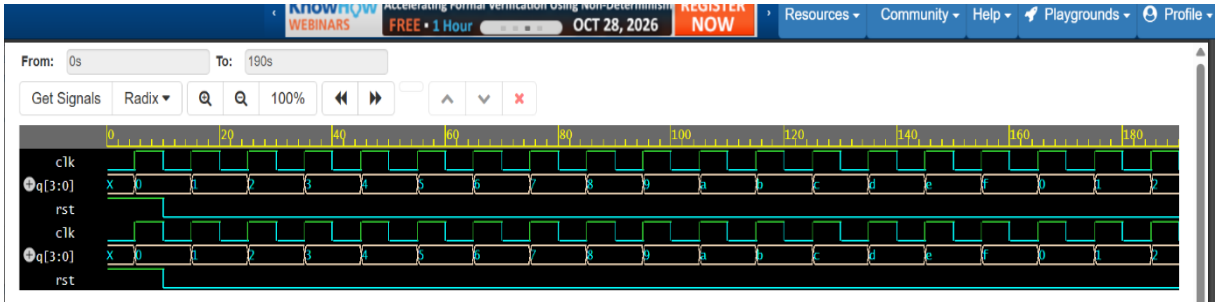


RESET initializes the state; outputs are Q[3:0].

Code



Output Waveform

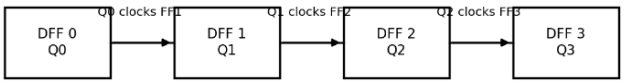


The waveform displays the clock, reset and four-bit output of the synchronous counter. Initially, reset sets the counter output to 0000. After reset is deactivated, all flip-flops respond to the same rising edge of the clock, and the binary count increases by one at each clock pulse. The counter progresses from 0000 to 1111 and then returns to 0000, demonstrating synchronous counting.

Asynchronous Counter

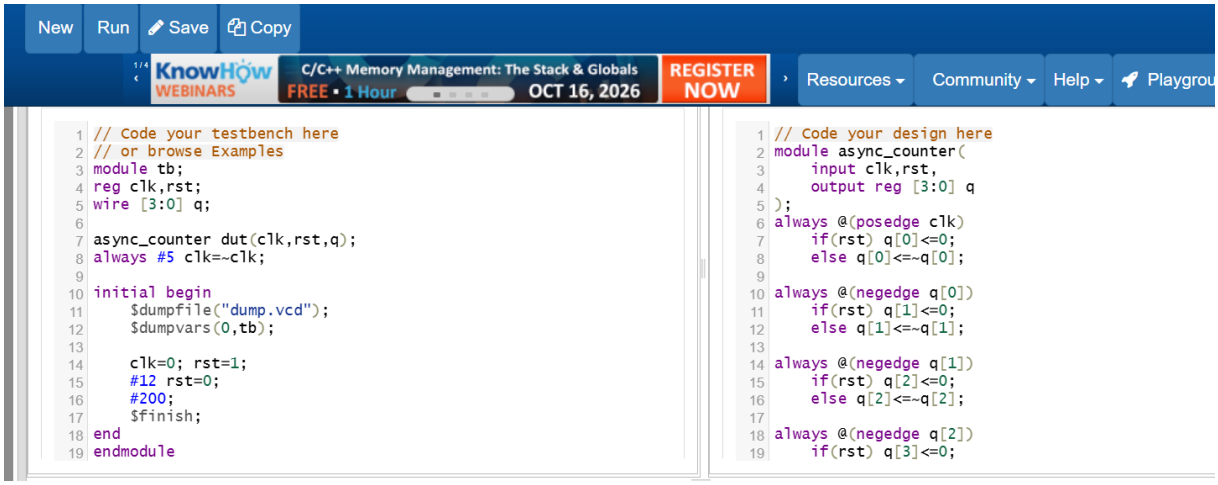
An asynchronous counter, also called a ripple counter, uses the output of one flip-flop to clock the next flip-flop. The binary count advances through the states as transitions ripple through the stages.

8B. 4-bit asynchronous ripple counter circuit



RESET initializes the state; outputs are Q[3:0].

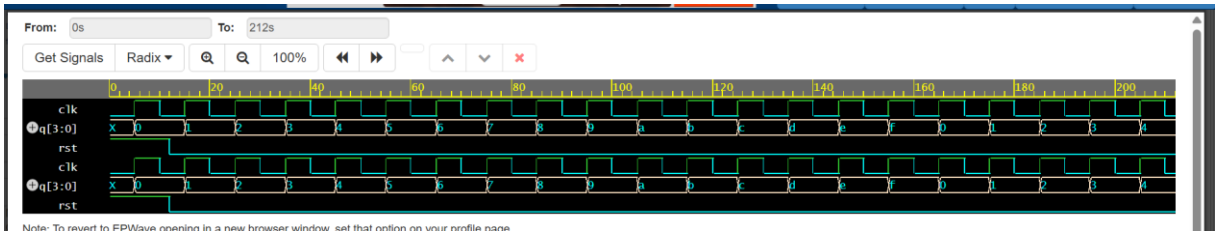
Code



Code

```
module async_counter(
    input clk,rst,
    output reg [3:0] q
);
always @(posedge clk)
    if(rst) q[0]<=0;
    else q[0]<=~q[0];
always @(negedge q[0])
    if(rst) q[1]<=0;
    else q[1]<=~q[1];
always @(negedge q[1])
    if(rst) q[2]<=0;
    else q[2]<=~q[2];
always @(negedge q[2])
    if(rst) q[3]<=0;
    else q[3]<=~q[3];
endmodule
```

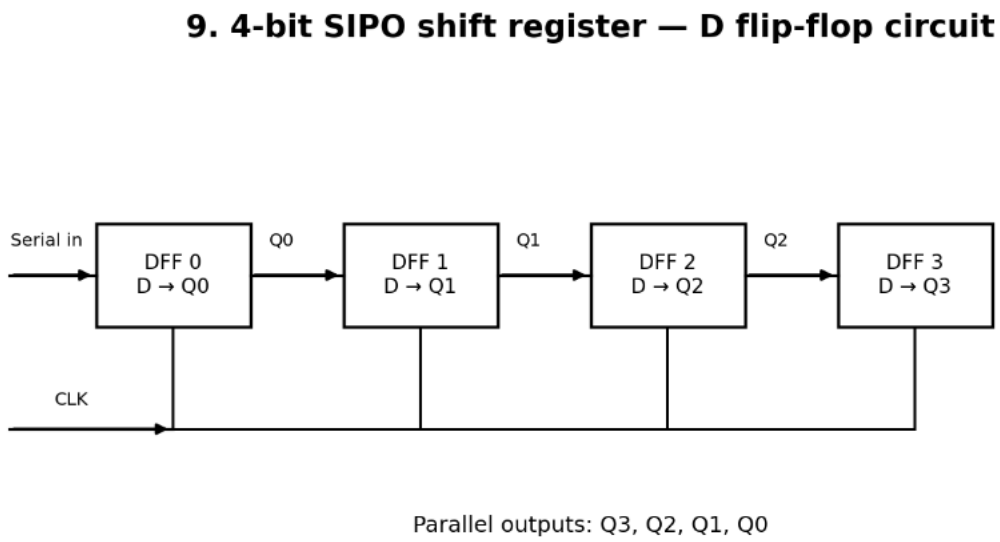
Output Waveform



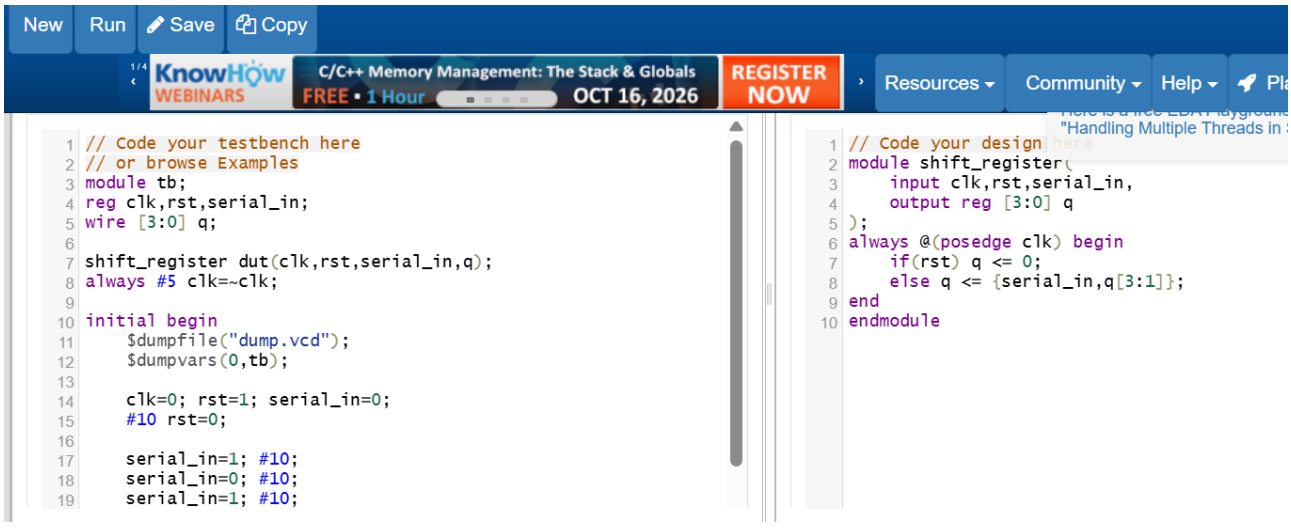
The waveform shows the clock, reset and four-bit output of the asynchronous ripple counter. Initially, reset clears all the flip-flops and sets the output to 0000. When clock pulses are applied, the first flip-flop toggles, and its output triggers the subsequent flip-flops to produce the binary count. The waveform demonstrates the ripple-counting operation, with small propagation delays between successive stages.

9. Shift Register

A shift register stores binary data using a series of flip-flops and shifts the data on each active clock edge. A serial-in, parallel-out register accepts serial input and provides its stored bits simultaneously at the outputs.



Code



Testbench

module tb;

reg clk,rst,serial_in;

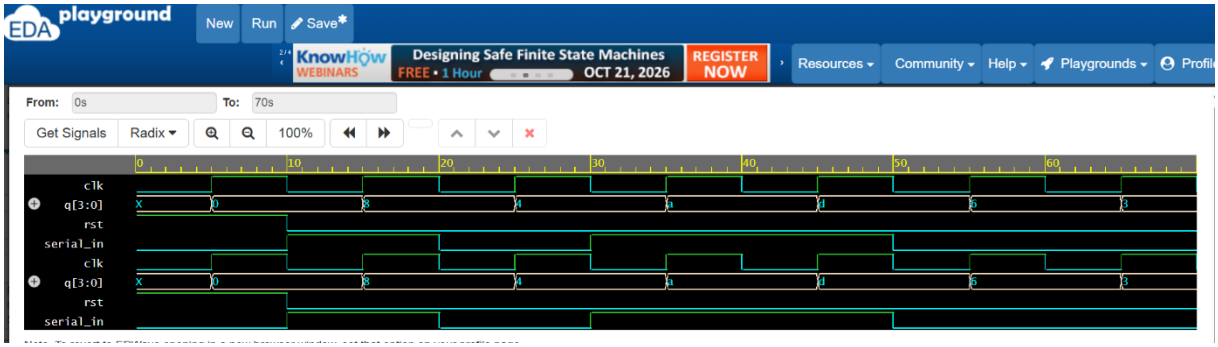
wire [3:0] q;

shift_register dut(clk,rst,serial_in,q);

always #5 clk=~clk;

```
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    clk=0; rst=1; serial_in=0;
    #10 rst=0;
    serial_in=1; #10;
    serial_in=0; #10;
    serial_in=1; #10;
    serial_in=1; #10;
    serial_in=0; #20;
    $finish;
end
endmodule
```

Output Waveform

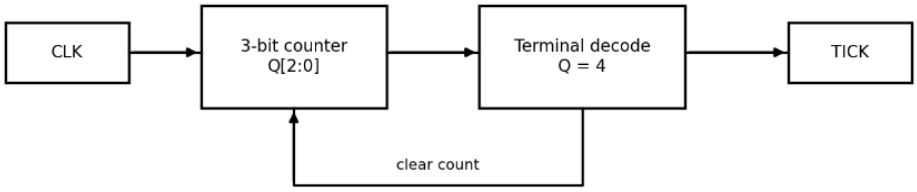


The waveform displays the clock, reset, serial input and four-bit parallel output. Initially, reset clears all the flip-flops, producing the output 0000. When serial data is applied, each rising clock edge shifts the stored bits to the next stage and inserts the new serial input bit. The waveform demonstrates how serial data is stored and made available simultaneously through the parallel outputs.

10. Divide-by-N-Counter

A divide-by-N counter counts a fixed number of clock cycles before returning to its initial state. In the modulo-5 example, it generates a tick pulse once every five clock cycles.

10. Divide-by-5 pulse generator — circuit



Code

```
// Code your testbench here
// or browse Examples
module tb;
reg clk,rst;
wire tick;

divide_n dut(clk,rst,tick);
always #5 clk=~clk;

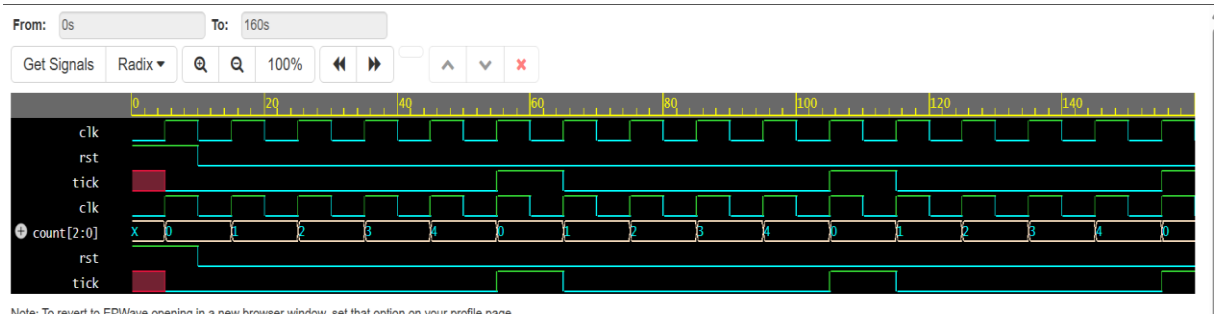
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);

    clk=0; rst=1;
    #10 rst=0;
    #150;
    $finish;
end
endmodule
```

```
// Code your design here
module divide_n(
    input clk,rst,
    output reg tick
);
reg [2:0] count;

always @(posedge clk) begin
    if(rst) begin
        count <= 0;
        tick <= 0;
    end
    else if(count == 4) begin
        count <= 0;
        tick <= 1;
    end
    else begin
        count <= count + 1;
        tick <= 0;
```

Output Waveforms

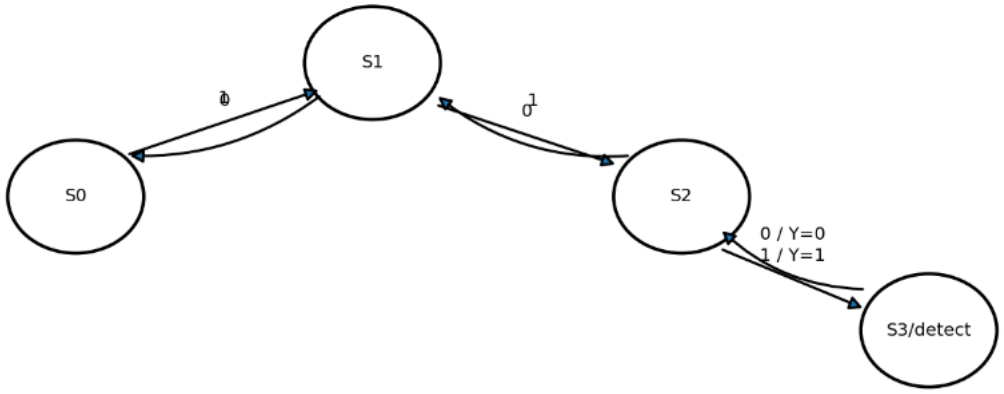


The waveform shows the clock, reset, internal counter value and tick output of the divide-by-5 circuit. Initially, reset clears the counter and sets the tick output to LOW. After reset is released, the counter progresses from zero to four and returns to zero after reaching the terminal count. The tick output becomes HIGH for one clock cycle every five clock cycles, verifying the divide-by-5 counting operation.

11. Mealy and Moore FSMs

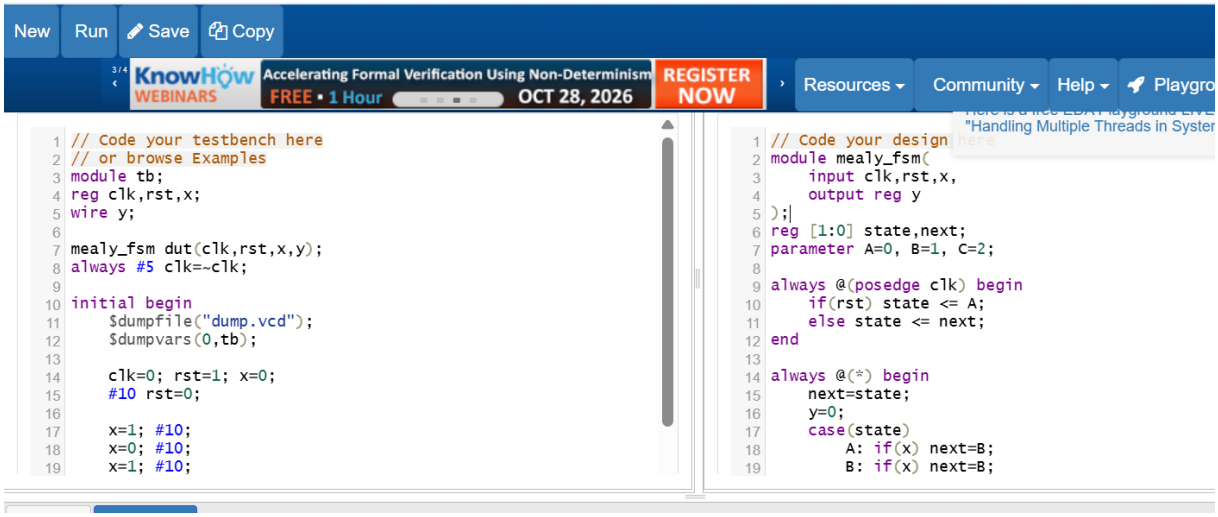
Mealy Finite State Machine (FSM)

A Mealy FSM is a sequential circuit whose output depends on both the current state and the current input. In the 101 sequence detector, the output becomes HIGH when the final input bit completes the sequence.



Mealy output is labelled on transitions as input / output.

Code



Code

```
module mealy_fsm(
    input clk, rst, x,
    output reg y
);
reg [1:0] state;
```

```
parameter S0=2'b00, S1=2'b01, S2=2'b10;
always @(posedge clk or posedge rst) begin
   if (rst)
      state <= S0;
   else begin
      case (state)
         S0: if (x) state <= S1;
            else   state <= S0;
         S1: if (x) state <= S1;
            else   state <= S2;
         S2: if (x) state <= S1;
            else   state <= S0;
         default: state <= S0;
      endcase
   end
end
always @(*) begin
   y = 0;
   if (!rst && state == S2 && x == 1)
      y = 1;
end
endmodule
```

Testbench

```
module tb;
reg clk, rst, x;
wire y;
mealy_fsm dut (
   .clk(clk),
   .rst(rst),
   .x(x),
```

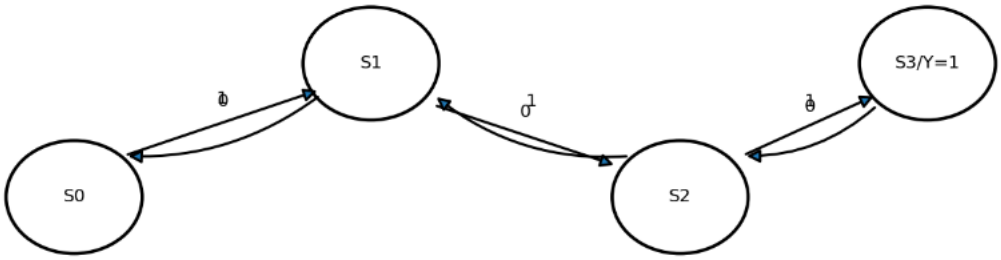
```
    .y(y)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);
    clk = 0;
    rst = 1;
    x = 0;
    #12 rst = 0;
    // Apply 101, then another 101
    @(negedge clk); x = 1;
    @(negedge clk); x = 0;
    @(negedge clk); x = 1;
    @(negedge clk); x = 0;
    @(negedge clk); x = 1;
    @(negedge clk); x = 0;
    @(negedge clk); x = 1;
    repeat (3) @(negedge clk);
    $finish;
end
endmodule
```

Output Waveform

The waveform displays the clock, reset, serial input, internal state and output of the Mealy FSM. The state changes according to the input sequence and the active clock edges. When the input sequence 101 is detected, the output becomes HIGH during the final input bit while the machine is in the appropriate detection condition. The waveform demonstrates sequence detection and the dependence of the Mealy output on both the current state and input.

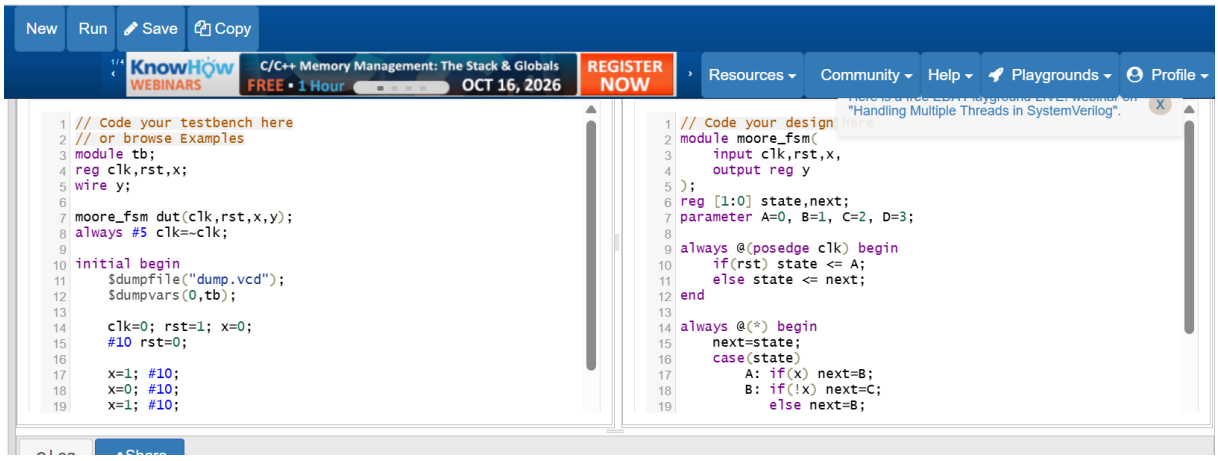
Moore Finite State Machine (FSM)

A Moore FSM is a sequential circuit whose output depends only on its current state. In the 101 sequence detector, a dedicated detection state produces the HIGH output.



Moore output: Y = 1 only in state S3; output is state-decoded.

Code



Design Code

```
module moore_fsm(
    input clk, rst, x,
    output reg y
);
reg [1:0] state;
parameter S0=2'b00, S1=2'b01,
          S2=2'b10, S3=2'b11;
always @(posedge clk or posedge rst) begin
```

```
    if (rst)
        state <= S0;
    else begin
        case (state)
            S0: if (x) state <= S1;
                else   state <= S0;
            S1: if (x) state <= S1;
                else   state <= S2;
            S2: if (x) state <= S3;
                else   state <= S0;
            S3: if (x) state <= S1;
                else   state <= S2;
            default: state <= S0;
        endcase
    end
end
always @(*) begin
    y = (state == S3);
end
endmodule
```

Testbench

```
module tb;
reg clk, rst, x;
wire y;
moore_fsm dut (
    .clk(clk),
    .rst(rst),
    .x(x),
    .y(y)
);
```

```
always #5 clk = ~clk;
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb);
   clk = 0;
   rst = 1;
   x = 0;
   #12 rst = 0;
   // Apply sequence 101
   @(negedge clk); x = 1;
   @(negedge clk); x = 0;
   @(negedge clk); x = 1;
   // Apply another sequence to verify overlap
   @(negedge clk); x = 0;
   @(negedge clk); x = 1;
   repeat (4) @(negedge clk);
   $finish;
end
endmodule
```

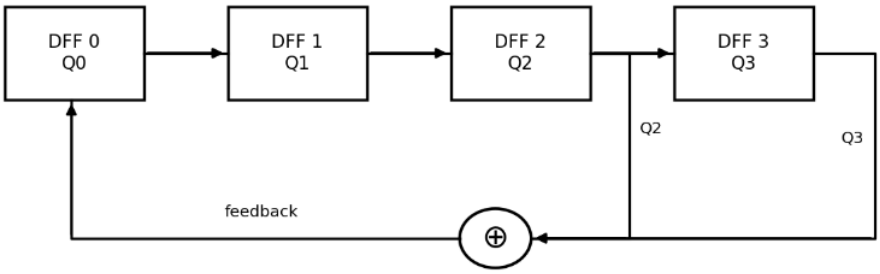
Output Waveforms

The waveform displays the clock, reset, serial input, internal state and output of the Moore FSM. The machine changes state at each active clock edge according to the input sequence. When the sequence 101 is detected, the FSM enters its dedicated detection state, causing the output to become HIGH. The waveform demonstrates that the Moore output depends only on the current state and remains HIGH while the machine stays in the detection state.

12. Random Sequence Generator

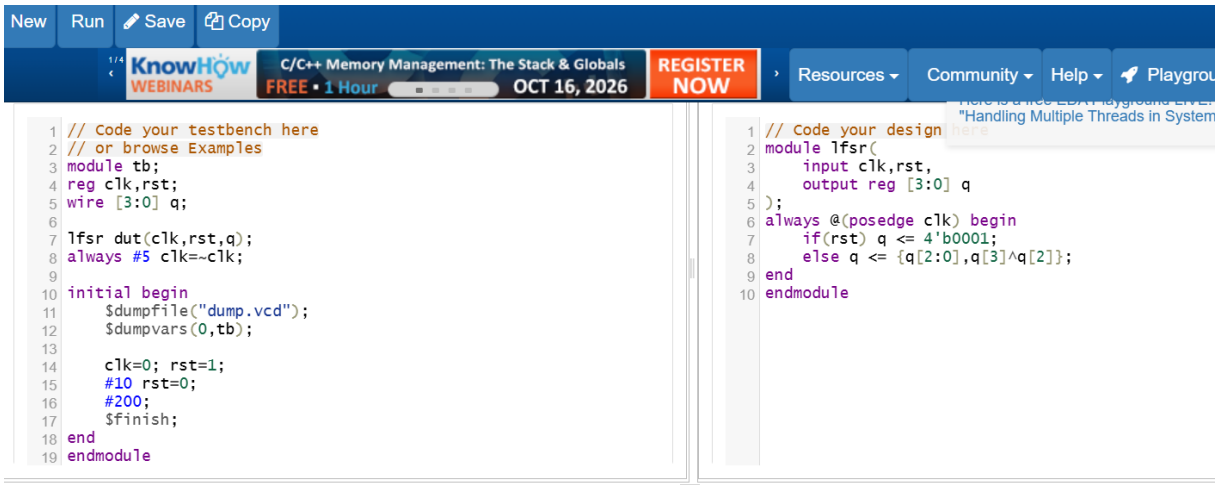
A linear-feedback shift register (LFSR) generates a pseudo-random sequence using flip-flops and XOR feedback logic. A non-zero initial seed is required to prevent the register from remaining permanently in the all-zero state.

12. 4-bit LFSR — feedback circuit



Seed must be non-zero (example: 0001).

Code



```
// Code your testbench here
// or browse Examples
module tb;
reg clk,rst;
wire [3:0] q;

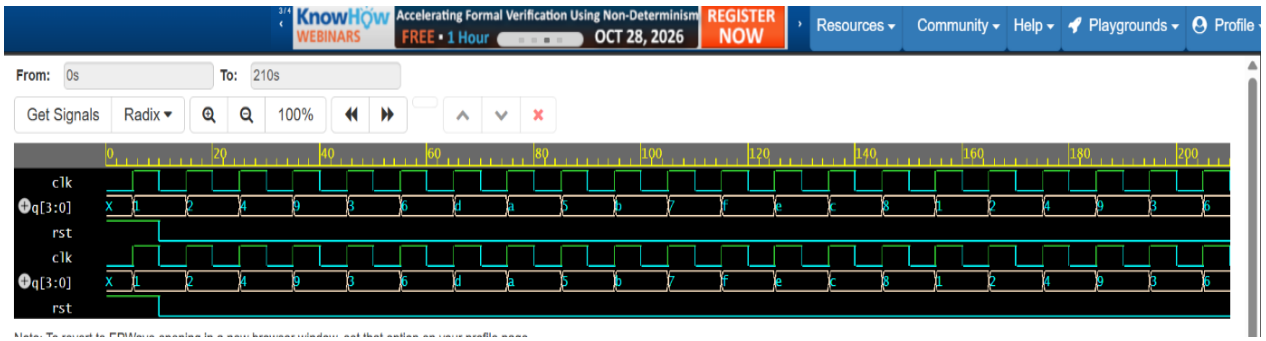
lfsr dut(clk,rst,q);
always #5 clk=~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);

    clk=0; rst=1;
    #10 rst=0;
    #200;
    $finish;
end
endmodule
```

```
// Code your design here
module lfsr(
    input clk,rst,
    output reg [3:0] q
);
always @(posedge clk) begin
    if(rst) q <= 4'b0001;
    else q <= {q[2:0],q[3]^q[2]};
end
endmodule
```

Output Waveform



The waveform shows the clock, reset and four-bit output of the linear-feedback shift register. Initially, reset loads the non-zero seed 0001 into the register. On every rising clock edge, the stored bits shift and the XOR feedback generates a new bit, producing a sequence of binary states. The waveform verifies the generation of a repeating pseudo-random sequence using the LFSR circuit.

Result

Thus, all the Verilog HDL programs were successfully designed and simulated, and their output waveforms were observed and verified.